

Q1. Explain how you would build an LLM-driven interview scoring system that is:

- Fair
- Consistent
- Non-biased
- Explainable

-> For fairness and non biasness, I would begin by masking the PII that can potentially add bias to the agent. Such PII may include name, gender, age or any other demographic information.

Also, it would be important to consider accent as a factor for biasness as well. The risks of a speech to text models incorrect interpretation due to difference in accent can cause the agent to assign lower points to candidate with higher value.

-> For consistency, the first idea is to ensure that the LLM is set to provide cold responses.

Focusing on factual correctness and assessment of the candidate rather than creativity.

For this, I would set the temperature parameter on the lower end around 0.1 to 0.3.

Second, I would verify the quality of each system prompt on user data. Only when the agent start providing similar responses to near similar user prompts it would be clear that the current system prompt is consistent and production ready.

-> For explainability, a dashboard which displays agent's scores with various parameter the user was tested on would be applicable. The dashboard displays the agent's questions. Along with the intended abilities tested and user's score on each of the intended ability.

Q2. Explain a method to detect fabricated answers or cheating patterns

(e.g., reading from screen, switching tabs, speaking pre-generated text).

-> To detect cheating patterns of fabricated approach a multi step approach would be required.

Reading Detection:

1. Candidate's eye movement will be tracked (Open Source example:- <https://github.com/NativeSensors/EyeGestures>).
2. After the interview, the eye tracking data will be parsed through a classifier model trained to detect reading movements
3. Or a clustering model to detect anomalous eye movements. The model can output a confidence score and higher confidence score for reading can be flagged for human verification.

Switching tabs:

1. Using JavaScript, we can get the current state of the window and tabs opened.
2. Additionally if the state changes to another tab being opened or switched, it can trigger a cheating attempt detected.

Pre-generated texts:

1. Using text-to-speech models, we will get the speech transcripts of the candidate.
2. The transcripts will then be checked using an AI detector (Open source example:- <https://github.com/bhaskatripathi/TypeTruth>)

Q3. Write Python code to:

- Extract embeddings for 1000 resumes
- Store them into a vector database
- Allow single JD → top 5 candidate search

Use FAISS or Pinecone.

-> Code in Section 1 folder

Q4. Describe how you would implement each of these during online interviews:

- Face detection
- Eye gaze tracking
- Tab change detection
- Voice consistency check
- Background noise analysis
- Multiple person detection

Explain your tech stack for each.

-> Face detection and multiple person detection can be solved using Object Detection models, such as Yolo. Yolo being a lightweight model can work on livestream to detect number of number of faces during the interview. Any number greater than 1 can be flagged for suspicious activity later to be checked by a human in the loop.

Also deepface models (Open source example:- <https://github.com/serengil/deepface>) will provide a confidence score between the facial features from the candidate's ID card and the live stream.

-> For eye tracking gaze, as previously mentioned models (Open source example:- <https://github.com/NativeSensors/EyeGestures>) for eye tracking will be used.

Then the eye tracking pattern will be analyzed using a clustering model to detect if the eye tracking was anomalous in nature (Detected as outlier).

-> Using JavaScript, we can get the current state of the window and tabs opened. Additionally if the state changes to another tab being opened or switched, it can trigger a cheating attempt detected.

-> Voice consistency will be checked using pre-trained models to analyze real-time audio features like MFCC coefficients, pitch, and energy against a baseline voice sample captured during interview intro to detect unnatural modulation or synthetic speech patterns.

-> Use pre-trained models to separate background noise with primary sound (Open source example:- <https://huggingface.co/speechbrain/mtl-mimic-voicebank>). Then the background noise can be analyzed separately.

Q5. Mini Coding Task

Write Python code to detect face presence from a webcam feed and raise an alert if face disappears for > 3 seconds.

-> Code in Section 2 folder

Q6. Explain the steps to measure sentiment, emotion, and tone from an audio answer.

-> Using speech to text models such as whisper or open source models like wav2vec we will get the clear transcripts of the audio. We will analyze the transcript and the audio separately.

The audio will be tested on factors like pitch, loudness, MLFCC. And the audio features will then be used to classify for nuance detection, emotion.

While the features from text and audio will be used together to measure the sentiment.

Q7. Describe how you would measure communication quality, including:

- Pace
- Filler words
- Pauses
- Energy
- Coherency

(Simple bullet points.)

-> For pace, we will compute speaking rate as syllables per second or words per minute from ASR transcript divided by speech duration.

-> For filler words, we will transcribe with Whisper, count fillers ("um", "uh", "like") using regex. Then compute filler words ratio to total words.

-> For pauses, we will detect silence with energy thresholding. We will then measure pause frequency/duration ratio to speech time. Example: More than 15% indicates hesitation.

Q8. Design an end-to-end pipeline KrantiQ uses when a candidate takes an interview:

1. Candidate starts
2. Proctoring
3. Question delivery (audio/video avatar)
4. Answer capture (audio/video)
5. Transcription
6. LLM scoring
7. Skill extraction
8. Similarity analytics
9. Report generation

Write in clear bullet points.

-> Candidate Starts: Candidate will access the KrantiQ interview platform via a secure web link, where initial face detection using DeepFace verifies identity against uploaded ID, establishing a baseline voice sample and monitoring setup.

Proctoring: Real-time monitoring activates with face detection (YOLO for multiple persons and away from keyboard), eye gaze tracking (e.g., EyeGestures for reading patterns), tab switching detection via JavaScript window state changes, voice consistency checks (MFCC, pitch against baseline), and background noise analysis (speechbrain models).

Question Delivery: Questions delivered via audio/video avatar. System prompts will ensure fair, consistent delivery with low LLM temperature (0.1-0.3) for factual focus, masking PII like name, gender, age, and accent biases.

Answer Capture: Candidate responses will be captured as audio/video streams. Continuous proctoring flags anomalies like face disappearance (e.g., 3-second alert via webcam feed), synthetic speech, or outliers in eye movements via clustering.

Transcription: Audio transcribed using Whisper or wav2vec2 for accurate text. Separate audio features (pitch, loudness, MFCC) extracted for emotion/tone analysis.

LLM Scoring: Transcripts fed to LLM with cold temperature (0.1-0.3) and verified prompts for consistent, non-biased scoring on factual correctness. PII masked to prevent demographic bias.

Skill Extraction: LLM analyzes transcripts for targeted abilities (e.g., pace as words/minute, filler words via regex on "um/uh/like", pauses via energy thresholding, coherency via sentiment models). Scores aggregated per parameter.

Similarity Analytics: Embeddings from transcripts compared for cheating (e.g., AI-generated text detection via TypeTruth). Anomalous patterns like pre-generated speech or outlier gaze flagged with confidence scores.

Report Generation: Dashboard compiles scores per ability/question, with explainable breakdowns, proctoring flags for human review, and visualizations of metrics like speaking rate.

Pause ratios (>15% hesitation), energy, and overall fairness.

Q9. Write a prompt that your LLM would use to score an interview answer.

-> You are a fair, consistent, and unbiased interview scoring agent for KrantiQ. Your role is to evaluate candidate answers based solely on factual correctness, technical depth, clarity, and relevance to the question. Ignore all demographic information, accent, name, gender, age, or any PII. Treat the response as anonymized text.

CRITERIA (score each 0-10, where 0=fail, 10=expert-level):

1. Factual Accuracy: Is the content correct and evidence-based?
2. Technical Depth: Does it demonstrate required skills/knowledge?
3. Clarity & Structure: Logical flow, concise, no fluff?
4. Completeness: Addresses all question aspects?
5. Communication Quality: Pace (words per min), low fillers (um/uh/like <5%), minimal pauses, high energy/coherency.

QUESTION: {question_text}

INTENDED SKILLS: {skills_list} (e.g., Python, problem-solving)

CANDIDATE ANSWER: {transcribed_answer}

AUDIO METRICS: Pace={pace_wpm} wpm, Fillers={filler_ratio}%, Pauses={pause_ratio}%,
Sentiment={sentiment_score}

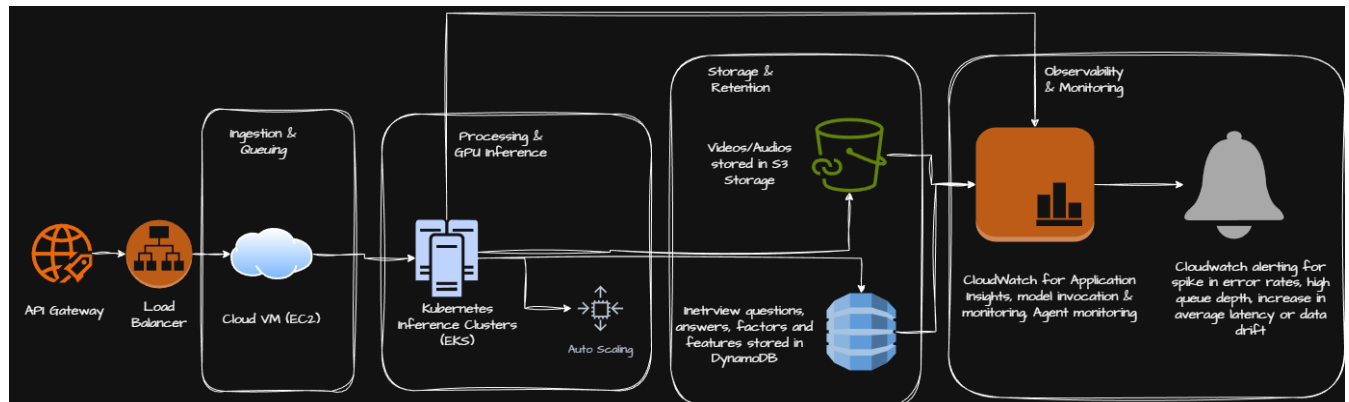
Provide ONLY this JSON output, no explanations:

```
{
  "overall_score": <0-10>,
  "criteria_scores": {
    "accuracy": <0-10>,
    "depth": <0-10>,
    "clarity": <0-10>,
    "completeness": <0-10>,
    "communication": <0-10>
  },
  "strengths": ["bullet1", "bullet2"],
  "improvements": ["bullet1", "bullet2"],
  "reasoning": "brief 1-sentence justification"
}
```

Q10. Design the architecture for:

- High-volume interview processing
- GPU inference pipeline
- Queue-based job processing
- Auto-scaling
- Logging & monitoring
- Storage & retention

->



Q11. Explain how you would reduce cloud costs for:

- Video processing
- Audio processing
- LLM scoring
- Embeddings
- Storage

-> For video processing, to avoid real time cloud GPU cost, a lightweight object detection model such as YOLO will run on client side, to detect multiple faces or missing face. And the anomalous

event's timestamp will be stored separately.

-> For audio processing, cpu optimized models, example Wav2Vec can run on serverless function example lambda, to run the audio models on-demand.

-> For LLM scoring, we will use models at low temperature and enable caching prompt templates to save cost on repetitive user responses.

-> For embeddings, we will use cpu efficient models or frameworks, example fastembed (As used in Section 1). Also reuse embeddings for same questions by the Agent.

-> For storage, we will compress the video files and reduce the fps to reduce storage after processing.

Q12. Write Python code to chunk long transcripts into overlapping segments of:

- Chunk size: 400 tokens
- Overlap: 80 tokens

-> Check code at Section 7/text_splitter.py

Q13. Write Python code to build a rate limiter allowing 20 API requests per minute per user.

-> Check code at Section 7/rate_limiter.py